Mastering ModusToolbox™ in Your Development Workflow — VS Code Edition

Lab Manual — Hands-On Exercises (VS Code Edition)

Training Session: [Date] | **Presenter:** [Name] **Prerequisites:** ModusToolbox™ 3.9 or later installed, ModusToolbox™ for VS Code extension installed, internet access for GitHub

---

Before You Begin

Prerequisites Checklist

Verify the following before starting the labs:

✅	Requirement	How to Verify
☐	**ModusToolbox 3.9 or later** installed	Open Modus-Shell — if it launches, you're good
☐	**Git** available in Modus-Shell	Run `git --version` — Git is included with the ModusToolbox installation
☐	**Internet access** to GitHub	Run `ping github.com` (required for `git clone` and `make getlibs`)
☐	**ModusToolbox™ for VS Code** extension installed	Open VS Code → Extensions view → search "ModusToolbox" → confirm it's installed and enabled
☐	(Optional) **KIT_PSC3M6_EVAL** eval board + USB cable	Required only for programming exercises

About Modus-Shell

Modus-Shell is a pre-configured terminal that ships with ModusToolbox on Windows operating systems. It sets up all the environment variables — compiler paths, tool locations, and build system paths — so that `make`, `git`, and ModusToolbox CLI tools work out of the box.

How to launch: Windows Start Menu → **Modus-Shell** (or find it in your ModusToolbox installation folder).

Lab 1: Clone, Resolve Dependencies & Build

Segment: 1 — Working Effectively with Version Control & Git **Tools:** Modus-Shell (command line only)

> 💡 **Lab 1 uses Modus-Shell exclusively.** All subsequent labs are done in VS Code using the ModusToolbox™ for VS Code extension.

Objective

Understand the Git + `make getlibs` workflow — clone a real ModusToolbox project from GitHub, resolve its dependencies, and build it. This is the foundational workflow for every ModusToolbox project you'll encounter in the field.

Background

ModusToolbox projects use a lightweight dependency system built on `.mtb` files — small text files that contain a Git URL and version tag pointing to a library. Here's what you need to know:

Concept	What It Means
`.mtb` **file**	A text file containing a Git URL + version tag pointing to a library.
`deps/` **directory**	YOUR declared dependencies — the `.mtb` files you (or Project Creator / Library Manager) put here. **Committed to Git.**
`bsps/` **directory**	YOUR BSP selection — the `.mtb` file for your target board, plus local BSP configuration files. **Committed to Git.**
`libs/` **directory**	Generated by `make getlibs`. Contains the resolved transitive `.mtb` files (dependencies of your dependencies) AND the generated `mtb.mk` makefile that wires everything into the build. **Never committed to Git.**
`mtb_shared/` **directory**	The actual cloned library source code. Lives ONE LEVEL ABOVE your project and is shared across all projects in the workspace. **Never committed to Git.**
`make getlibs`	The dependency resolver. Reads all `.mtb` files in `deps/` and `bsps/`, resolves the full dependency tree, clones source code into `mtb_shared/`, and generates the build wiring in `libs/`.

`deps/` and `bsps/` contain your declared dependencies — the `.mtb` files you control. `make getlibs` resolves the full dependency tree, generates the build wiring in `libs/`, and clones the actual source code into `mtb_shared/`. You commit your declarations; everything else is reconstructed.

Step-by-Step Instructions

Step 1 — Open Modus-Shell

Launch Modus-Shell from your Windows Start Menu. You should see a terminal prompt with the ModusToolbox environment configured.

Step 2 — Create the training workspace

```
mkdir C:/MTB_Training && cd C:/MTB_Training
```

Expected result: You are now in an empty `C:/MTB_Training/` directory.

Step 3 — Clone the training project

```
git clone https://github.com/ClarkJ-Infineon/mtb-training-blinky
```

Expected result: Git clones the repository. You should see output like:

```
Cloning into 'mtb-training-blinky'...
remote: Enumerating objects: ...
Receiving objects: 100% ...
```

> ⚠️ **Notice:** There is no `mtb_shared/` or `libs/` directory yet — only the source code and dependency declarations were committed to Git.

Step 4 — Enter the project directory

```
cd mtb-training-blinky
```

Step 5 — Inspect the declared dependencies

```
ls deps/
```

Expected result: You should see the project's direct dependency `.mtb` files (e.g., `retarget-io.mtb`). Each file is a simple text pointer to a Git repository and version. Note that the BSP is in `bsps/`, not `deps/`.

To see what's inside an `.mtb` file:

```
cat deps/retarget-io.mtb
```

Expected result: A line containing a URL, version tag, and local path — for example:

```
https://github.com/cypresssemiconductorco/retarget-io#release-v1.9.0#$$ASSET_REPO$$/retarget-io/release-v1.9.0
```

Step 6 — Resolve dependencies with `make getlibs`

```
make getlibs
```

Expected result: The tool reads all `.mtb` files in `deps/`, clones each library into `../mtb_shared/`, and generates `libs/mtb.mk`. You'll see Git clone progress for each dependency. This may take 1–3 minutes depending on your internet connection.

When complete, you should see output ending with something like:

```
Library Manager: Completed
```

Step 7 — Inspect the `libs/` directory

```
ls libs/
```

Expected result: You should see many more `.mtb` files than were in `deps/` — these are the **transitive dependencies** (dependencies of your dependencies). You'll also see `mtb.mk` and other generated `.mk` files.

Step 8 — Inspect `libs/mtb.mk`

```
cat libs/mtb.mk
```

Expected result: A series of `SEARCH_*` variable definitions, each pointing to a library path under `../mtb_shared/`. These paths are how the build system finds library source code and headers. For example:

```makefile
SEARCH_retarget-io=../mtb_shared/retarget-io/release-v1.9.0
SEARCH_mtb-dsl-psc3m6=../mtb_shared/mtb-dsl-psc3m6/release-v1.0.0
...
```

Step 9 — Build the project

```
make build
```

Expected result: The GCC ARM compiler builds the project targeting `KIT_PSC3M6_EVAL`. Build output scrolls by, and the final lines should indicate success:

```
================================================================================
= Build complete =
================================================================================
```

The build output (`.elf`, `.hex` files) is placed in the `build/` directory.

Results Summary

Step	What You Should See
Clone	Project files appear — `Makefile`, `main.c`, `deps/`, `bsps/`
`ls deps/`	A few `.mtb` files (direct dependencies)
`make getlibs`	Libraries cloned, `libs/mtb.mk` generated
`ls libs/`	Many `.mtb` files (transitive deps) + `mtb.mk`
`cat libs/mtb.mk`	`SEARCH_*` paths pointing to `../mtb_shared/`
`make build`	Successful build with zero errors

Exploration Questions

1. **Transitive dependencies:** Notice that `deps/` has only a few `.mtb` files, but `libs/` has many more — those are transitive dependencies resolved by the build system. What would happen if you deleted `libs/` and `mtb_shared/` and ran `make getlibs` again?
2. **`.mtb` file contents:** Look at the `.mtb` file contents — what three pieces of information does each file contain? (Hint: URL, version, local path.)
3. **Reproducibility:** If a colleague clones this same repo on a different machine and runs `make getlibs` → `make build`, will they get the same result? Why or why not?

Lab 2: Open the Project in VS Code and Export with the Archiving Tool

Segment: 2 — Sharing, Importing & Exporting Projects **Tools:** ModusToolbox™ for VS Code extension

Objective

Open the cloned project in VS Code and create a smart archive using the extension's archiving and sharing tool. This demonstrates the two most common project-sharing workflows: IDE collaboration and archive-based handoff.

Background

Opening in VS Code: ModusToolbox projects are IDE-agnostic — they're built with Makefiles, not IDE project files. VS Code opens any folder directly; there is no separate "import" step. The ModusToolbox™ for VS Code extension detects the project automatically and surfaces build/program/configuration actions through its **Application** tab.

mtbarchive: A ModusToolbox-aware archiving tool that creates self-contained project ZIPs. Unlike a plain ZIP, `mtbarchive` knows which files are authored (include them) and which are generated (exclude them). The result is a small, clean archive that any recipient can restore with `make getlibs` → `make build`.

✅ Included by mtbarchive	❌ Excluded by mtbarchive
Source code, Makefile, configs	`libs/` (regenerated by `make getlibs`)
`deps/` (`.mtb` dependency declarations)	`build/` output
`bsps/` (BSP configuration & device config)	`mtb_shared/` (regenerated)
`.git/` history (if present)	IDE workspace files
README, LICENSE	

Prerequisites

- **Lab 1 completed** — project cloned, dependencies resolved, and built in `C:/MTB_Training/mtb-training-blinky`

Step-by-Step Instructions

Step 1 — Open the project in VS Code

From Modus-Shell, in your project directory:

```
code .
```

Or launch VS Code directly and use **File** → **Open Folder...** to select `C:/MTB_Training/mtb-training-blinky`.

Expected result: VS Code opens with the project visible in the **Explorer** panel — source files, BSP, and dependency structure are all visible immediately.

> 💡 **No import step required.** Unlike Eclipse, VS Code doesn't register or copy the project into a separate workspace — it works directly against the folder on disk. Your command-line builds and IDE actions operate on the same files.

Step 2 — Verify the ModusToolbox environment

Click the **ModusToolbox™ for VS Code** icon in the editor title bar (or open the Command Palette with `Ctrl+Shift+P` and run **ModusToolbox™: Show Main Page**), then select the **Application** tab.

Expected result: The Application tab shows your project path and the detected **Toolsdir** (e.g., `C:/Users/<user>/ModusToolbox/tools_3.9`).

> 💡 **No version-association step needed.** Unlike Eclipse, which requires you to explicitly associate a project with an installed ModusToolbox version, the extension auto-detects the Toolsdir from your environment. Just confirm it shows the expected version.

Step 3 — Verify the project in VS Code Explorer

Expand the project in Explorer. You should see:

- `main.c` (your application code)
- `deps/` (your `.mtb` dependency declarations)
- `bsps/` (BSP configuration files)
- `libs/` (resolved dependencies — generated)

- `Makefile`

Step 4 — Build from the Application tab

In the Application tab, click **Build**.

Expected result: The build output streams into the VS Code output/terminal panel, ending with a successful build message — the same result you saw from the command line in Lab 1.

Step 5 — Launch the archiving and sharing tool

In the Application tab, scroll down to the **Tools** section and click **ModusToolbox™ archiving and sharing**.

Expected result: The mtbarchive GUI opens with the current project pre-selected.

> 💡 **CLI alternative:** The `mtbarchive` command is not on the PATH by default in Modus-Shell or the IDE terminal. If you prefer the CLI, use the full path:
>
> ```
> C:/Users/<user>/ModusToolbox/tools_3.9/mtbarchive/mtbarchive.exe -v -o project_archive.zip -f .
> ```

Step 6 — Export the archive

In the mtbarchive GUI, use the default settings and click **Export** (or **Create Archive**).

Expected result: An archive ZIP file is created. Note the file size — it should be approximately **1–3 MB**.

Step 7 — Inspect the archive contents

Open the archive with any ZIP viewer (or extract it to a temporary folder). Verify the contents include:

- ✅ Source code (`main.c` , etc.)
- ✅ `deps/` with `.mtb` files
- ✅ `bsps/` with BSP configuration
- ✅ `Makefile`
- ❌ No `libs/` directory
- ❌ No `build/` directory
- ❌ No `mtb_shared/` directory

Results Summary

Step	What You Should See
Open in VS Code	Project visible in Explorer immediately — no import step
Application tab	Project path and Toolsdir auto-detected
Project structure	Source, `deps/` , `bsps/` , `libs/` , `Makefile` all visible
Build from Application tab	Successful build (same as command line)
mtbarchive export	ZIP file ~1–3 MB
Archive contents	Source + deps + bsps — no libs, build, or mtb_shared

Exploration Questions

1. **Archive size:** Compare the archive size (~1–3 MB) to the total project size on disk after building (check with `du -sh .` or look at folder properties). Why is the archive so much smaller?
2. **Recipient workflow:** What would a recipient need to do after unzipping this archive to get a working, buildable project? (Hint: two commands.)
3. **Git vs. mtbarchive:** When would you recommend a customer use Git for sharing versus `mtbarchive`? What are the trade-offs?

Lab 3: Create a Custom BSP

Segment: 4 — Workflows to Develop & Utilize a Custom BSP **Tools:** ModusToolbox™ for VS Code extension (Library Manager, Device Configurator)

Objective

Create a custom Board Support Package (BSP) from the kit BSP, modify pin assignments, and build with the new BSP. This is the exact workflow you'll guide customers through when they move from an eval board to their production hardware.

Background

Why custom BSPs matter: Infineon kit BSPs are a starting point — they provide a working configuration for the eval board. But most production designs have different pin assignments, crystal frequencies, debug interfaces, and memory layouts. Creating a dedicated custom BSP gives the customer a clean baseline for their specific hardware.

The workflow:

Step	Action
1	Copy the closest kit BSP
2	Rename to `TARGET_<your_name>`
3	Set it as the active BSP (via Library Manager)
4	Edit `design.modus` for your hardware (Device Configurator)
5	Build, test, iterate

Key BSP files:

File	Purpose	Tool
`config/design.modus`	Pin assignments, clock tree, peripheral config	Device Configurator
`config/GeneratedSource/`	Auto-generated C code from `design.modus`	Device Configurator (auto)
`cybsp.h`	Board-level macros (`CYBSP_USER_LED`, `CYBSP_DEBUG_UART`, etc.)	Auto-generated by Device Configurator
`cybsp.c`	Board initialization implementation	Text editor (rarely modified)

File	Purpose	Tool
`linker/*.ld`	Memory regions	Text editor (advanced)

> ⚠️ **Important:** Do not manually edit files in `GeneratedSource/` — they are regenerated from `design.modus` every time you save in Device Configurator. Your edits would be overwritten.

Prerequisites

- **Lab 1 completed** — project cloned and built in `C:/MTB_Training/mtb-training-blinky`

Step-by-Step Instructions

Step 1 — Copy the kit BSP to create a new BSP

In the **VS Code Explorer**, expand `bsps/` in your project. Right-click `TARGET_APP_KIT_PSC3M6_EVAL` → **Copy**, then right-click `bsps/` → **Paste**. Rename the pasted folder to `TARGET_MY_BOARD`.

Expected result: A new directory `bsps/TARGET_MY_BOARD/` appears in Explorer, containing an exact copy of the kit BSP — `config/design.modus`, `cybsp.h`, `cybsp.c`, `GeneratedSource/`, linker scripts, and all other BSP files.

> 💡 **CLI alternative (VS Code integrated terminal):** You can also do this from the terminal within VS Code (see Appendix A for terminal setup):
>
> ```
> cd C:/MTB_Training/mtb-training-blinky  cp -r bsps/TARGET_APP_KIT_PSC3M6_EVAL bsps/TARGET_MY_BOARD
> ```

> 💡 **Naming convention:** Custom BSP directories always start with `TARGET_`. Use a descriptive name — e.g., `TARGET_MY_PRODUCT_V2`, `TARGET_CUSTOMER_MAIN_BOARD`.

Step 2 — Set the new BSP as active using Library Manager

In the **Application** tab (ModusToolbox™ for VS Code Main Page), click **Configure Middleware**.

> 💡 **CLI shortcut (VS Code terminal):** You can also launch Library Manager from the integrated terminal by running `make modlibs`.

In Library Manager:

1. Find `MY_BOARD` in the BSP list
2. Select it as the **active BSP**
3. Click **Update**

Expected result: Library Manager updates the Makefile `TARGET=` variable to `MY_BOARD`, regenerates `libs/mtb.mk`, and resolves dependencies for the new BSP.

Two ways to switch the active BSP:

Option A — Library Manager GUI (recommended)

Open via **Configure Middleware** in the Application tab. Select the BSP → click **Update**. Library Manager handles everything in one step.

Option B — Makefile edit + make getlibs (CLI alternative)

1. Open `Makefile` and change `TARGET=APP_KIT_PSC3M6_EVAL` to `TARGET=MY_BOARD`
2. Run `make getlibs` to regenerate `libs/mtb.mk` with the new BSP path

Step 3 — Open Device Configurator on the new BSP

In the **Application** tab, click **Configure Device**.

Expected result: Device Configurator opens as a separate window, showing the inherited kit configuration — all the pin assignments, clock settings, and peripheral configurations from the original `KIT_PSC3M6_EVAL` BSP — targeting whichever BSP is currently active (the one you set in Step 2).

> ⚠️ **Unlike Eclipse, double-clicking `design.modus` in the VS Code Explorer does not launch Device Configurator** — it just opens the file as plain XML text. Always launch Device Configurator through the **Configure Device** button in the Application tab.

Step 4 — Change one pin assignment

In Device Configurator, modify a pin assignment — for example, move `USER_LED` to a different GPIO pin:

1. Find the current `USER_LED` pin assignment in the Pins tab
2. Disable or reassign it
3. Assign `USER_LED` to a different available GPIO pin

Expected result: The pin assignment updates visually in Device Configurator. The old pin is freed, and the new pin shows the `USER_LED` assignment.

Step 5 — Save the configurator changes

Click **Save** in Device Configurator (or press `Ctrl+S`).

Expected result: Device Configurator regenerates all files in `config/GeneratedSource/`:

- `cycfg_pins.h` / `cycfg_pins.c` — updated pin mux configuration
- `cycfg_clocks.h` / `cycfg_clocks.c` — clock tree configuration
- `cycfg_peripherals.h` / `cycfg_peripherals.c` — peripheral configuration
- `cycfg_system.h` / `cycfg_system.c` — system/power configuration

You can verify the pin change by opening `config/GeneratedSource/cycfg_pins.c` and looking for the new pin assignment.

Step 6 — Build with the custom BSP

In the **Application** tab, click **Build**.

Expected result: The project builds successfully with your custom `MY_BOARD` BSP. The build output should show `TARGET=MY_BOARD` and complete with zero errors.

```
================================================================================
= Build complete =
================================================================================
```

> 💡 **CLI alternative:** `make build` from Modus-Shell or the VS Code integrated terminal produces the same result.

Step 7 — (Optional) Program and verify

If you have a `KIT_PSC3M6_EVAL` eval board connected, click **Program** in the Application tab.

Expected result: The firmware is flashed to the board, and the LED blinks on the new pin (if you wired an LED there, or if the new pin has an on-board LED connected).

> 💡 **CLI alternative:** `make program` from Modus-Shell or the VS Code integrated terminal produces the same result.

Results Summary

Step	What You Should See
Copy BSP	`bsps/TARGET_MY_BOARD/` directory with full BSP contents
Library Manager (Configure Middleware)	`MY_BOARD` set as active BSP, Makefile updated
Device Configurator (Configure Device)	Inherited kit configuration, editable
Pin change	New pin assignment visible in configurator
Save	`GeneratedSource/` files regenerated (including `cybsp.h`)
Build (Application tab)	Successful build with `TARGET=MY_BOARD`

Exploration Questions

1. **Customer scenario:** A customer sends you their custom board schematic. What's the first thing you do to create their BSP? What questions do you ask the customer before starting?

   Consider:

   - Which eval board did they base their design on?
   - Which silicon variant (MPN) are they using?
   - What pins are reassigned compared to the eval board?
   - What external components are different (crystal, flash, debug interface)?

2. **BSP Assistant vs. manual copy:** When would you use BSP Assistant instead of a manual copy? (Hint: think about changing the target device/MPN.)

3. **Sharing the BSP:** If this custom BSP needs to be shared across multiple projects in a team, what are your options? (Hint: project-local, shared Git repo, or manifest-published.)

Appendix A: ModusToolbox Command Reference

A quick reference for the most commonly used ModusToolbox commands and their VS Code extension equivalents.

Build Commands

Command	VS Code Button (Application tab)	What It Does	When to Use
`make build`	**Build**	Compile the project for the active BSP and toolchain	After code changes, to verify everything compiles
`make clean`	**Clean**	Delete all build artifacts (`build/` directory)	When builds behave unexpectedly, or before a fresh build

Command	VS Code Button (Application tab)	What It Does	When to Use
`make program`	**Program**	Build (if needed) and flash firmware to connected board	To deploy and test on hardware
`make getlibs`	*(no dedicated button — see note below)*	Download/update all dependencies from `.mtb` files	After cloning a repo, or when dependencies change
—	**Rebuild**	Clean + Build in one step	Equivalent to `make clean build`
—	**Erase**	Erase the connected board's flash	Before reprogramming, or to confirm a board is blank

> 💡 **`make getlibs` in VS Code:** The extension resolves dependencies automatically the first time it loads an application (you'll see a **"Preparing Application For VS Code"** dialog run `make vscode`, which includes dependency resolution). If you add or change a `.mtb` file afterward, use **Configure Middleware** (Library Manager) to re-resolve, or run `make getlibs` from the integrated terminal.

IDE Integration Commands

Command	What It Does	When to Use
`make vscode`	Generate VS Code settings and `.code-workspace` file	The ModusToolbox™ for VS Code extension runs this automatically the first time you open the Application tab for a project; run it manually only if you're not using the extension

Tool Launcher Commands

Command	VS Code Button (Application tab)	What It Does	When to Use
`make modlibs`	**Configure Middleware**	Launch Library Manager	Adding, updating, or removing middleware libraries
`make bsp-assistant`	**BSP Assistant** tile (Tools section)	Launch BSP Assistant	Creating or modifying a BSP (especially MPN changes)
—	**Configure Device**	Launch Device Configurator	Editing pin assignments, clocks, and peripheral config

Standalone Tools

The Application tab's **Tools** section (scroll down from Build/Program) provides one-click launchers for every standalone ModusToolbox tool, including:

Tool Tile	What It Does	When to Use
ModusToolbox™ archiving and sharing	Launch `mtbarchive` GUI	Sharing a project via email, USB, or support ticket
BSP Assistant	Create/modify BSPs, including MPN changes	Switching silicon variant or starting a new custom BSP
Device Firmware Update Host Tool	Launch the DFU host utility	Field firmware updates over UART/I2C/USB/BLE

Tool Tile	What It Does	When to Use
Smart I/O Configurator	Configure Smart I/O routing	Hardware-based logic between GPIOs without firmware
ModusToolbox™ Motor Suite	Motor control tuning GUI	Motor control application development
ModusToolbox™ Programmer	Standalone flash/erase/verify tool	Programming outside the context of a specific project
Edge Protect Configurator	Secure boot / lifecycle configuration	PSOC Edge security provisioning workflows

> 💡 **CLI alternative:** `mtbarchive` is not available directly in Modus-Shell or the IDE terminal. Use the full path:
>
> `C:/Users/<user>/ModusToolbox/tools_3.9/mtbarchive/mtbarchive.exe -v -o <file.zip> -f <project>`

Switching the Active BSP (CLI)

Edit the Makefile `TARGET=` variable, then run `make getlibs` to regenerate `libs/mtb.mk`:

```
# 1. Edit Makefile: change TARGET=APP_KIT_PSC3M6_EVAL to TARGET=MY_BOARD
# 2. Regenerate dependency paths
make getlibs
```

Useful Options

Option	Applies To	What It Does
`TOOLCHAIN=GCC_ARM`	`make build`	Specify compiler (default: `GCC_ARM`; also `ARM`, `IAR`)
`CONFIG=Release`	`make build`	Build in Release mode (default: `Debug`)
`TARGET=<BSP_NAME>`	`make build`	Override the active BSP target (use Library Manager instead)

Using Modus-Shell from VS Code

You can run ModusToolbox CLI commands directly from within VS Code without switching to a separate Modus-Shell window.

1. Open the integrated terminal (**Terminal** → **New Terminal** or `Ctrl+``)
2. Click the **+** (dropdown arrow) next to the terminal tab to see available launch profiles
3. Select **ModusToolbox™ Shell** from the list
4. A new terminal opens with the ModusToolbox environment configured

> 💡 The **ModusToolbox™ Shell** terminal profile is provided by the **ModusToolbox™ for VS Code** extension. If it does not appear, verify the extension is installed and enabled.

Appendix B: Troubleshooting

`make getlibs` Fails

Symptom	Cause	Fix
`fatal: unable to access` or timeout errors	No internet / corporate proxy blocking GitHub	Check network connection; configure Git proxy: `git config --global http.proxy http://proxy:port`
`Permission denied (publickey)`	SSH key not configured for GitHub	Use HTTPS URLs in `.mtb` files (default), or configure SSH keys
`make: *** No rule to make target 'getlibs'`	Not in a ModusToolbox project directory, or Modus-Shell not configured	Verify you're in the project root (where `Makefile` is); restart Modus-Shell

Build Fails After Switching BSPs

Symptom	Cause	Fix
Missing header files (`No such file or directory`)	`libs/mtb.mk` is stale — still pointing to old BSP's paths	Run `make getlibs` to regenerate, or use Library Manager (**Configure Middleware**) to switch BSPs
`TARGET` mismatch errors	Makefile `TARGET=` doesn't match BSP directory name	Use Library Manager (not manual edit) to switch BSPs
Linker errors referencing wrong memory regions	Stale build artifacts from previous BSP	Click **Clean** then **Build** in the Application tab

Missing Headers After Clone

Symptom	Cause	Fix
`fatal error: cybsp.h: No such file or directory`	Dependencies not resolved	Open the project in VS Code (triggers automatic resolution) or run `make getlibs` then **Build**
`fatal error: cy_pdl_config.h: No such file or directory`	Same — dependencies not resolved	Same fix
Build works on one machine but not another	Different library versions or missing dependency resolution	Ensure both machines open the project in VS Code (or run `make getlibs`); pin versions in `.mtb` files

Device Configurator Issues

Symptom	Cause	Fix
Double-clicking `design.modus` opens it as text instead of launching Device Configurator	VS Code has no built-in file association for `.modus` files	Always launch via **Configure Device** in the Application tab
Changes in Device Configurator don't take effect	Forgot to save, or editing wrong BSP	Verify you're editing the **active** BSP's `design.modus` ; save (`Ctrl+S`)
`GeneratedSource/` files look wrong or outdated	Stale generated files	Re-save `design.modus` in Device Configurator to force regeneration
`cybsp.h` macros don't match pin config	`cybsp.h` not updated after Device Configurator change	Manually update `cybsp.h` macros to match new pin assignments

General Tips

- **When in doubt, clean and rebuild:** Click **Clean**, then **Build** in the Application tab (or `make clean && make getlibs && make build` from the terminal).
- **Check `libs/mtb.mk` exists:** If it's missing, dependency resolution hasn't run yet.
- **Verify active BSP:** Open `Makefile` and check the `TARGET=` line — it should match your intended BSP directory name (without the `TARGET_` prefix in the Makefile, but WITH `TARGET_` in the `bsps/` directory name).
- **Read the library README:** If a newly added library doesn't work, check `mtb_shared/<library>/<version>/README.md` for required `COMPONENTS` and `DEFINES`.

Appendix C: Key Concepts Reference

The ModusToolbox Project Structure

```
my-project/
├── Makefile                    ← Project build configuration
├── main.c                      ← Your application code
├── deps/                       ← YOUR declared dependencies (committed to Git)
│   └── retarget-io.mtb
├── bsps/                       ← BSP files (local to project)
│   └── TARGET_APP_KIT_PSC3M6_EVAL/
│       ├── cybsp.c / cybsp.h   ← Board init API
│       └── config/
│           └── design.modus    ← Device Configurator master file
├── libs/                       ← Resolved dependency tree (generated, never commit)
│   ├── mtb.mk                  ← Generated SEARCH paths
│   └── (transitive .mtb files)
├── build/                      ← Build output (never commit)
└── mtb_shared/                 ← SHARED library source (one level up, never commit)
    ├── mtb-dsl-psc3m6/
    │   └── release-v1.0.0/
    ├── retarget-io/
    │   └── release-v1.9.0/
    └── ...
```

`deps/` vs `libs/` vs `mtb_shared/` — Know the Difference

	`deps/` + `bsps/`	`libs/`	`mtb_shared/`
What's inside	`.mtb` files you declared (direct dependencies + BSP)	Resolved transitive `.mtb` files + generated `mtb.mk` build wiring	Actual cloned library source code
Who controls it	You (the developer)	`make getlibs` (the tool)	`make getlibs` (the tool)
Commit to Git?	✅ Yes — these are your declarations	❌ No — reconstructed by `make getlibs`	❌ No — reconstructed by `make getlibs`
Scope	Per-project	Per-project	Per-workspace (shared across projects)

What to Commit, What to Ignore

✅ COMMIT	❌ IGNORE
`Makefile`	`build/`
`main.c` and all source files	`mtb_shared/`
`deps/*.mtb`	`libs/`
`bsps/` (BSP customizations)	`.vscode/`
`.gitignore`	`.cache/`

End of Lab Manual — VS Code Edition